1. Class, Object & Reference Variable

BCA study notes | Simple English • Java examples • Diagrams

Class

A **class** is a blueprint or template that defines data (fields) and actions (methods). It describes what its objects will contain and do.

Object

An **object** is a real instance of a class. Every object has its own instance data and can use the methods defined by the class.

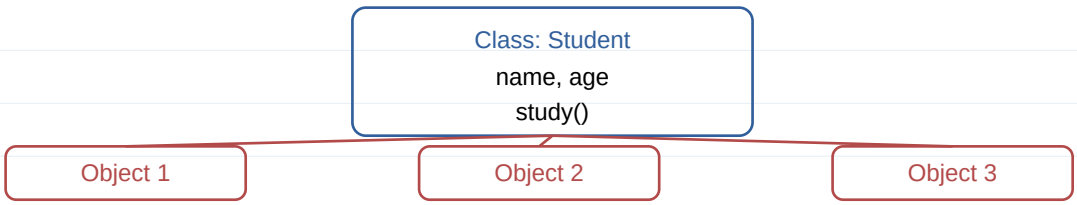
Reference variable

A reference variable stores the reference to an object. It does not contain the complete object itself.

Java code example

```
class Student {
    String name;
    void study() {
        System.out.println(name + " is studying");
    }
}
Student s1 = new Student(); // object + reference
s1.name = "Asha";
s1.study();
```

Concept diagram



2. new Keyword & Instance Members

BCA study notes | Simple English • Java examples • Diagrams

new keyword

The **new** operator creates an object in memory, initializes its instance fields to default values, and invokes a constructor.

Instance variables

Instance variables are declared inside a class but outside methods. Each object gets its own copy.

Instance methods

Instance methods belong to objects. They can directly access instance variables and use the **this** reference.

Java code example

```
class Mobile {
    String brand;       // instance variable
    void call() {       // instance method
        System.out.println(brand);
    }
}
Mobile m = new Mobile();
m.brand = "Samsung";
m.call();
```

3. Default & Parameterized Constructors

BCA study notes | Simple English • Java examples • Diagrams

Constructor

A constructor is a special member used to initialize a new object. Its name is the same as the class name and it has no return type, not even void.

Default constructor

If no constructor is written, Java provides a no-argument default constructor. If you write any constructor, Java does not automatically add that default constructor.

Parameterized constructor

A constructor with parameters lets us provide initial values when creating an object.

Java code example

```
class Student {
    String name;
    int age;
    Student() {             // no-argument constructor
        name = "Unknown";
        age = 0;
    }
    Student(String n, int a) { // parameterized
        name = n;
        age = a;
    }
}
Student s1 = new Student();
Student s2 = new Student("Ravi", 20);
```

Concept diagram

4. Constructor Overloading & Chaining

BCA study notes | Simple English • Java examples • Diagrams

Overloading

A class can have multiple constructors with different parameter lists. This is called constructor overloading.

Chaining

Constructor chaining means calling one constructor from another constructor using **this()** or calling a parent constructor using **super()**.

Rule

A this() or super() constructor call must be the first statement in a constructor. You cannot use both as separate constructor calls in the same constructor.

Java code example

```
class Box {
    int size;
    Box() {
        this(1);          // calls parameterized constructor
    }
    Box(int size) {
        this.size = size;
    }
}
Box b = new Box();
```

5. this & this()

BCA study notes | Simple English • Java examples • Diagrams

this

The keyword **this** refers to the current object. It is often used when a parameter name is the same as an instance variable.

this()

The expression **this()** calls another constructor of the same class. It is used to avoid repeating initialization code.

Difference

this refers to the current object; this() invokes another constructor. They are related but not interchangeable.

Java code example

```
class Employee {
    String name;
    Employee(String name) {
        this.name = name; // field = parameter
    }
    Employee() {
        this("Not given"); // constructor call
    }
}
```

6. Static Members & Static Blocks

BCA study notes | Simple English • Java examples • Diagrams

Static members

A static variable or method belongs to the class, not to each individual object. One static variable is shared by all objects of that class.

Static method

Call it using the class name, for example Counter.show(). It cannot directly use instance fields or this.

Static block

A static initialization block runs once when the class is initialized. It is commonly used to initialize static data.

Java code example

```
class Counter {
    static int count;
    static {
        count = 100;       // runs once
    }
    Counter() {
        count++;
    }
}
Counter c1 = new Counter();
Counter c2 = new Counter();
System.out.println(Counter.count);
```

7. Instance Initialization Blocks

BCA study notes | Simple English • Java examples • Diagrams

Meaning

An instance initialization block is a block of code inside a class, outside methods and constructors. It runs whenever an object is created.

Order

For each object, instance field initializers and instance blocks run in their textual order, before the constructor body.

Use

It is useful when multiple constructors need to share common initialization code. Constructor chaining affects how many times initialization occurs: once per object construction.

Java code example

```
class Demo {
    int x = 5;
    {
        x = x + 2;        // instance block
        System.out.println("Block: " + x);
    }
    Demo() {
        System.out.println("Constructor: " + x);
    }
}
new Demo();
```

8. Initialization Order

BCA study notes | Simple English • Java examples • Diagrams

Class initialization

Static fields and static blocks run once, in textual order, when the class is initialized.

Object initialization

When an object is created: memory is allocated and fields get default values; superclass initialization occurs; then this class's instance field initializers and instance blocks run in textual order; then the constructor body runs.

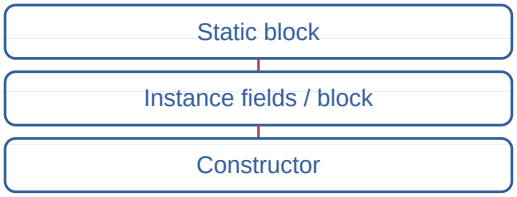
Remember

Static initialization happens once per class initialization. Instance initialization happens for each new object.

Java code example

```
class Order {
    static { System.out.println("1. static"); }
    { System.out.println("2. instance block"); }
    Order() { System.out.println("3. constructor"); }
}
new Order();
new Order();
```

Concept diagram



9. Encapsulation & Data Hiding

BCA study notes | Simple English • Java examples • Diagrams

Encapsulation

Encapsulation means combining data and methods in one class and controlling how the data is accessed.

Data hiding

Data hiding protects internal data by declaring fields **private**, so outside code cannot access them directly.

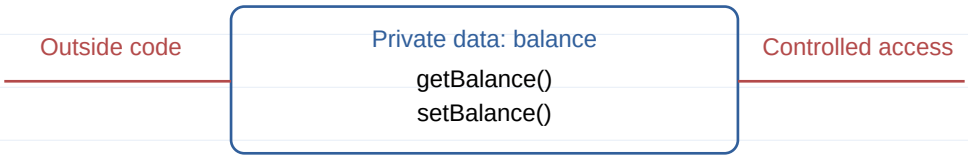
Getters and setters

A getter returns a private field's value. A setter updates it, and can validate the new value before storing it.

Java code example

```
class Account {
    private double balance;
    public double getBalance() {
        return balance;
    }
    public void setBalance(double amount) {
        if (amount >= 0) balance = amount;
    }
}
Account a = new Account();
a.setBalance(500);
System.out.println(a.getBalance());
```

Concept diagram



10. Quick Revision & Complete Example

BCA study notes | Simple English • Java examples • Diagrams

Quick points

Class = blueprint; object = instance; reference = object reference; new = creates object; constructor = initializes object.

Constructors

Same name as class, no return type; can be overloaded; this() calls another constructor in the same class.

Access control

Use private fields and public getters/setters to support encapsulation and data hiding.

Java code example

```
class Student {
    private String name;
    private int age;
    static String college = "SRMU";

    Student() {
        this("Unknown", 0);
    }
    Student(String name, int age) {
        this.name = name;
        this.age = age;
    }
    public String getName() { return name; }
    public void setAge(int age) {
        if (age >= 0) this.age = age;
    }
    public static void main(String[] args) {
        Student s = new Student("Asha", 19);
        s.setAge(20);
        System.out.println(s.getName());
        System.out.println(Student.college);
    }
}
```

Quick revision table

Term	Meaning
Class	Blueprint / template
Object	Instance of a class
this	Current object reference
this()	Calls another constructor
static	Belongs to the class
private	Accessible inside the declaring class